Lock-Free Concurrency in Java

Concurrency problems often begin with a simple shared variable.

```
class Counter {

    private int count = 0;

    public void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

The method looks like one operation, but `count++` is conceptually a read-modify-write sequence:

```
int current = count;     // read
int updated = current + 1;
count = updated;         // write
```

If two threads execute these steps at the same time, one update can overwrite the other.

```
Initial count = 10

Thread A reads 10
Thread B reads 10

Thread A calculates 11
Thread B calculates 11
```

```
Thread A writes 11
Thread B writes 11
```

Two increments occurred, but the final value is only `11`.

This is a **race condition** caused by a non-atomic compound operation.

---

1. Atomicity

An operation is atomic when other threads cannot observe or interfere with a partially completed version of that operation.

```
Atomic operation
=
all-or-nothing operation
```

From the perspective of other threads, the operation appears to happen as one indivisible step.

This does not necessarily mean that only one machine instruction is involved. It means the operation has a single visible effect under the Java Memory Model.

---

2. Traditional Lock-Based Solution

The counter can be protected using `synchronized`.

```java
class Counter {

    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
```

```
    }
}
```

A lock ensures that only one thread executes the protected critical section at a time.

```
Thread A acquires lock
        ↓
Thread B waits
        ↓
Thread A updates count
        ↓
Thread A releases lock
        ↓
Thread B continues
```

Locks are necessary when several related operations or variables must be protected as one critical section.

For small updates to a single value, Java also provides atomic classes.

---

3. What Does Lock-Free Mean?

A lock-free algorithm does not require threads to acquire a traditional mutual-exclusion lock before making progress.

Threads usually:

1. read the current state
2. calculate a desired new state
3. attempt an atomic update
4. retry if another thread changed the state first

```
Read
  ↓
Calculate
  ↓
Compare and update
```

```
   ├── Success → finish
   └── Failure → read latest state and retry
```

Lock-free does **not** mean:

- there is no coordination between threads
- contention disappears
- every thread completes immediately
- lock-free code is always faster
- threads never wait in any form

It means that system-wide progress is possible without one thread owning a traditional lock that every other thread must acquire.

---

4. Progress Guarantees

The word “non-blocking” covers several different guarantees.

Blocking

A thread may wait for another thread that owns a lock.

```
Thread A owns lock
Thread B cannot continue until A releases it
```

Lock-Free

At least one competing thread is guaranteed to make progress.

An individual thread may repeatedly lose the race and retry.

Wait-Free

Every participating thread is guaranteed to complete within a bounded number of steps.

Wait-free algorithms provide a stronger guarantee than lock-free algorithms and are harder to design.

Obstruction-Free

A thread makes progress when it eventually runs without interference from other threads.

For most application-level Java code, the practical distinction is between traditional lock-based coordination and atomic lock-free updates.

Part 1: Atomic Variables

5. The `java.util.concurrent.atomic` Package

Java provides atomic classes for safely updating individual values.

Common classes include:

```
AtomicInteger
AtomicLong
AtomicBoolean
AtomicReference<V>
AtomicIntegerArray
AtomicLongArray
AtomicReferenceArray<E>
AtomicStampedReference<V>
AtomicMarkableReference<V>
```

These classes provide:

- atomic reads and writes
- atomic read-modify-write operations
- visibility guarantees

- compare-and-set operations
- functional update methods

---

6. `AtomicInteger`

`AtomicInteger` stores an integer that can be updated atomically by multiple threads.

```
import java.util.concurrent.atomic.AtomicInteger;
```

Thread-Safe Counter

```
import java.util.concurrent.atomic.AtomicInteger;

class Counter {

    private final AtomicInteger count =
            new AtomicInteger(0);

    public void increment() {
        count.incrementAndGet();
    }

    public int getCount() {
        return count.get();
    }
}
```

Test it with two threads:

```
public class AtomicCounterDemo {

    public static void main(String[] args)
            throws InterruptedException {
```

```
        Counter counter = new Counter();

        Thread firstThread = new Thread(() -> {
            for (int i = 0; i < 1_000; i++) {
                counter.increment();
            }
        });

        Thread secondThread = new Thread(() -> {
            for (int i = 0; i < 1_000; i++) {
                counter.increment();
            }
        });

        firstThread.start();
        secondThread.start();

        firstThread.join();
        secondThread.join();

        System.out.println(counter.getCount());
    }
}
```

Output:

```
2000
```

Each increment is performed atomically, so one thread cannot overwrite another thread's completed increment.

---

7. Important `AtomicInteger` Methods

`get()`

Returns the current value.

```
int value = count.get();
```

`set(int newValue)`

Sets a new value.

```
count.set(100);
```

`incrementAndGet()`

Increments the value and returns the updated value.

```
int updated = count.incrementAndGet();
```

Conceptually similar to:

```
++value
```

`getAndIncrement()`

Returns the old value and then increments it.

```
int previous = count.getAndIncrement();
```

Conceptually similar to:

```
value++
```

`decrementAndGet()`

Decrements the value and returns the updated value.

```
int updated = count.decrementAndGet();
```

Conceptually similar to:

```
--value
```

`getAndDecrement()`

Returns the old value and then decrements it.

```
int previous = count.getAndDecrement();
```

Conceptually similar to:

```
value--
```

`addAndGet(int delta)`

Adds a value and returns the updated result.

```
int updated = count.addAndGet(5);
```

`getAndAdd(int delta)`

Returns the old value and then performs the addition.

```
int previous = count.getAndAdd(5);
```

`getAndSet(int newValue)`

Atomically replaces the value and returns the previous value.

```
int previous = count.getAndSet(0);
```

`compareAndSet(int expected, int update)`

Updates the value only when its current value equals the expected value.

```
boolean changed =
        count.compareAndSet(10, 20);
```

8. Functional Update Methods

Atomic classes can calculate a new value using a function.

`updateAndGet()`

```
int updated = count.updateAndGet(
        current -> current * 2
);
```

The method atomically applies the function and returns the updated value.

`getAndUpdate()`

```
int previous = count.getAndUpdate(
        current -> current * 2
);
```

It returns the old value.

`accumulateAndGet()`

```
int updated = count.accumulateAndGet(
        5,
        Integer::sum
);
```

The current value and supplied argument are combined atomically.

Important Rule

The function supplied to an atomic update method may execute more than once when contention causes retries.

Therefore, it should be:

- side-effect free
- deterministic
- inexpensive

Avoid this:

```
count.updateAndGet(current -> {
    sendNotification(); // unsafe side effect
    return current + 1;
});
```

The notification could be sent multiple times even though the final update happens once.

---

9. `AtomicLong`

`AtomicLong` provides the same style of operations for `long` values.

```
import java.util.concurrent.atomic.AtomicLong;

class IdGenerator {

    private final AtomicLong sequence =
            new AtomicLong(0);

    public long nextId() {
        return sequence.incrementAndGet();
    }
}
```

Possible uses include:

- counters
- sequence numbers
- statistics
- locally generated identifiers
- total bytes processed

An atomic increment only guarantees uniqueness within that particular `AtomicLong` instance and process. It is not automatically a distributed ID generator.

---

10. `AtomicBoolean`

`AtomicBoolean` stores a boolean value with atomic operations.

```
import java.util.concurrent.atomic.AtomicBoolean;
```

Example:

```
class Service {

    private final AtomicBoolean running =
            new AtomicBoolean(true);

    public void stop() {
        running.set(false);
    }

    public void runTask() {
        while (running.get()) {
            performWork();
        }
    }

    private void performWork() {
        // Work
```

```
    }
}
```

For a simple stop flag, a `volatile boolean` can also provide the required visibility:

```
private volatile boolean running = true;
```

The main reason to use `AtomicBoolean` is when an atomic state transition is needed.

---

11. Atomic State Transition With `AtomicBoolean`

Suppose only one thread should initialise a resource.

```
class Initializer {

    private final AtomicBoolean initialized =
            new AtomicBoolean(false);

    public boolean initialize() {

        if (!initialized.compareAndSet(false, true)) {
            return false;
        }

        performInitialization();
        return true;
    }

    private void performInitialization() {
        System.out.println("Initialized once");
    }
}
```

Several threads may call `initialize()`, but only one can successfully change:

```
false → true
```

This pattern is sometimes called **test-and-set**.

Failure Consideration

If initialization can fail, changing the flag before the work completes may leave the object marked as initialized incorrectly.

A more complete state model may be required:

```
NOT_STARTED
INITIALIZING
INITIALIZED
FAILED
```

An `AtomicReference<State>` is better when more than two states exist.

---

12. Why Not Always Use `synchronized`?

Both approaches can be correct.

Use an atomic variable when:

- one independent value is being updated
- the operation can be expressed as one atomic transition
- lock-free progress is useful
- the critical operation is very small

Use a lock when:

- several fields must change together
- multiple steps must be protected as one invariant
- a condition must be awaited
- a collection of actions must be isolated
- the operation is easier to understand as a critical section

Correctness is more important than avoiding locks.

---

13. Atomic Variables Are Not Magic

An atomic class protects atomic operations performed through that class.

It does not automatically make a sequence of separate operations atomic.

This code is unsafe:

```
if (count.get() > 0) {
    count.decrementAndGet();
}
```

The logic contains two independent operations:

```
1. Check whether count is positive
2. Decrement count
```

Two threads can both pass the check before either decrements.

```
Initial count = 1

Thread A checks: 1 > 0
Thread B checks: 1 > 0

Thread A decrements to 0
Thread B decrements to -1
```

The individual methods are atomic, but the compound rule is not.

Part 2: AtomicReference

16. What Is `AtomicReference<V>`?

`AtomicReference<V>` performs atomic operations on an object reference.

```
import java.util.concurrent.atomic.AtomicReference;
```

It protects replacement of the reference.

It does not automatically make the referenced object's internal mutable fields thread-safe.

```
AtomicReference<List<String>> reference =
        new AtomicReference<>(new ArrayList<>());
```

Atomically replacing the list reference is safe.

Mutating the same `ArrayList` from several threads is still unsafe.

---

17. Basic `AtomicReference` Operations

```
AtomicReference<String> reference =
        new AtomicReference<>("Hello");

System.out.println(reference.get());

reference.set("World");

System.out.println(reference.get());
```

Output:

```
Hello
World
```

Atomic replacement:

```
String oldValue =
        reference.getAndSet("Java");
```

Conditional replacement:

```
boolean changed =
        reference.compareAndSet(
                "Java",
                "Concurrency"
        );
```

18. Seat Booking Race Condition

Consider one theatre seat:

```
class SeatBooking {

    private String seat = "EMPTY";

    public boolean bookSeat(String person) {

        if (seat.equals("EMPTY")) {
            seat = person;
            return true;
        }

        return false;
    }
}
```

Two threads can both observe the seat as empty:

```
Thread A reads EMPTY
Thread B reads EMPTY

Thread A writes Aditya
Thread B writes Rahul
```

Both methods return `true`, but only one name remains in the field.

This is a classic check-then-act race.

---

19. Seat Booking With `AtomicReference`

```
import java.util.concurrent.atomic.AtomicReference;

class SeatBooking {

    private final AtomicReference<String> seat =
            new AtomicReference<>("EMPTY");

    public boolean bookSeat(String person) {
        return seat.compareAndSet(
                "EMPTY",
                person
        );
    }

    public String getBookedBy() {
        return seat.get();
    }
}
```

If two threads execute the CAS operation:

```
Thread A: CAS(EMPTY → Aditya) → succeeds
Thread B: CAS(EMPTY → Rahul)  → fails
```

Only one transition can observe and replace the expected value atomically.

---

20. Reference Equality in `compareAndSet()`

For `AtomicReference`, `compareAndSet()` compares object references using identity semantics.

Conceptually:

```
currentReference == expectedReference
```

It does not call:

```
currentReference.equals(expectedReference)
```

This matters for custom objects.

```
record User(String name) {}

AtomicReference<User> reference =
        new AtomicReference<>(
                new User("Aditya")
        );

User expected =
        new User("Aditya");

boolean changed =
        reference.compareAndSet(
                expected,
                new User("Rohit")
        );
```

Even though the two `User` objects are logically equal, they are different object references. The CAS operation fails.

The expected value should normally be the exact reference previously obtained using `get()`.

Quick Revision

67. Core Terms

Term	Meaning
Race condition	Result depends on uncontrolled thread timing
Atomic operation	Appears indivisible to other threads
Lock-free	System-wide progress without traditional lock ownership
Wait-free	Every thread completes within bounded steps
CAS	Replace a value only when it still matches the expected value
Retry loop	Reread, recalculate and retry after CAS failure
ABA problem	Value changes and later returns to its original appearance
Atomic reference	Atomically replaces an object reference
Immutable state	State represented by an object that is never modified after construction
Contention	Several threads compete to update the same shared state

68. Atomic Classes Summary

Class	Purpose
`AtomicInteger`	Atomic integer operations
`AtomicLong`	Atomic long operations
`AtomicBoolean`	Atomic boolean state transitions
`AtomicReference<V>`	Atomic object-reference replacement
`AtomicIntegerArray`	Atomic operations on integer array elements
`AtomicLongArray`	Atomic operations on long array elements
`AtomicReferenceArray<E>`	Atomic reference operations per array index
`AtomicStampedReference<V>`	Reference plus integer version
`AtomicMarkableReference<V>`	Reference plus boolean marker
`LongAdder`	High-throughput counter under contention